



UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PERIODO : Abril – Agosto

ASIGNATURA : Fundamentos Web

TEMA : Construcción de interacciones web

ESTUDIANTE : Anahi Correa

NIVEL-PARALELO - NRC: Cuarto NRC 29535

DOCENTE : Ing. Geovanny Brito

FECHA DE ENTREGA : 16 de Julio del 2026

SANTO DOMINGO – ECUADOR

Índice

1. Enlace GitHub:	4
2. Título del taller.....	4
3. Descripción de la actividad.....	4
4. Objetivos.....	4
a. Objetivo General.....	4
b. Objetivos Específicos.....	4
5. Marco Conceptual	4
6. Desarrollo paso a paso	5
5.1 Agregar una cuarta nota al formulario	5
5.2 Permitir notas con decimales.....	5
5.3 Ajustar las carreras del formulario.....	6
5.4 Crear mensaje de beca académica	6
5.5 Mostrar colores según el tipo de beca.....	7
5.6 Calcular promedio especial sin la nota más baja	7
5.7 Mostrar nota más alta y nota más baja	7
5.8 Contar notas aprobadas, en recuperación y reprobadas	8
5.9 Generar recomendación académica	9
5.10 Clasificar el rendimiento académico.....	9
5.11 Crear historial de estudiantes evaluados	10
5.12 Validar nombres repetidos	10
5.13 Crear botón para listar estudiantes.....	10
5.14 Mostrar cuántos estudiantes han sido evaluados.....	11
5.15 Contar aprobados, recuperación y reprobados del curso	11
5.16 Calcular promedio general del curso.....	12
5.17 Buscar estudiante por nombre	12
5.18 Crear ranking de estudiantes.....	13
5.19 Mostrar estudiante con mayor y menor promedio	13
5.20 Mejorar el objeto final del estudiante.....	14
7. Capturas de la página web	15
8. Pruebas realizadas.....	16
9. Análisis de resultados	17
10. Conclusiones	18

11.	Recomendaciones	18
12.	Criterio personal	18
13.	Bibliografía complementaria	18

Índice de Figuras

Figura 1.	Evaluador Académico	15
Figura 2.	Historial de estudiantes	16
Figura 3.	Ranking de estudiantes	16
Figura 4.	Búsqueda de estudiante por nombre	16
Figura 5.	Validación de datos vacíos	17
Figura 6.	Validación cuando se ingresa el mismo nombre	17

Índice de Código

Código 1.	Campo Nota 4 en el formulario	5
Código 2.	Configuración de notas con decimales	6
Código 3.	Selector de carreras	6
Código 4.	Visualización del mensaje de beca	7
Código 5.	Colores según el tipo de beca	7
Código 6.	Cálculo del promedio sin la nota más baja	7
Código 7.	Obtención de la nota mayor y menor	8
Código 8.	Conteo de notas aprobadas, recuperación y reprobadas	8
Código 9.	Generación de la recomendación académica	9
Código 10.	Clasificación del rendimiento académico	9
Código 11.	Creación del historial de estudiantes	10
Código 12.	Validación de nombres repetidos	10
Código 13.	Listado de estudiantes en una tabla	11
Código 14.	Estudiantes evaluados	11
Código 15.	Conteo de estados del curso	12
Código 16.	Cálculo del promedio general del curso	12
Código 17.	Búsqueda de estudiantes por nombre	13
Código 18.	Generación del ranking de estudiantes	13
Código 19.	Estudiante con mayor y menor promedio	14
Código 20.	Creación del objeto estudiante en formato JSON	14

1. Enlace GitHub:

https://github.com/Anahi292/U3Taller1_CorreaAnahi_FundamentosWeb

2. Título del taller

CONSTRUCCIÓN DE INTERACCIONES WEB

3. Descripción de la actividad

El presente taller consistió en ampliar un evaluador académico que ya teníamos construido en clase, partiendo de los archivos base index.html y app.js. Sobre estos se agregaron nuevas funcionalidades como por ejemplo la validación de datos, el cálculo de promedios, la clasificación del rendimiento académico, el cálculo de becas, el manejo de un historial de estudiantes mediante arreglos y objetos, la generación de un ranking, la búsqueda de estudiantes y la mejora de la interfaz visual utilizando componentes de Bootstrap.

4. Objetivos

a. Objetivo General

Ampliar la lógica y la interfaz del evaluador académico mediante funciones, eventos, validaciones, arreglos, objetos, estructuras condicionales, ciclos y manipulación del DOM, aplicando componentes visuales de Bootstrap para mejorar la experiencia del usuario.

b. Objetivos Específicos

- Desarrollar un sistema de evaluación académica que permita registrar cuatro calificaciones por estudiante, calcular promedios, identificar la nota más alta y más baja, y clasificar las notas según su estado.
- Implementar funcionalidades para el análisis del rendimiento estudiantil, incluyendo la asignación de carreras, el cálculo del porcentaje de beca, la clasificación del desempeño y la generación de recomendaciones académicas.
- Diseñar un historial de estudiantes que permita almacenar registros sin duplicados, mostrar un ranking por promedio y generar estadísticas generales del curso para facilitar el análisis de resultados.

5. Marco Conceptual

El desarrollo del taller se apoya en conceptos fundamentales de JavaScript aplicados al navegador. Las funciones permiten encapsular la lógica de cálculo (promedio, clasificación de estado, cálculo de beca) en bloques reutilizables. Los eventos, gestionados mediante addEventListener, conectan las acciones del usuario (clics en

botones) con la lógica del programa. La manipulación del DOM, a través de métodos como `document.getElementById`, `classList` y `innerHTML`, permite leer los valores ingresados por el usuario y actualizar dinámicamente el contenido visual sin recargar la página. Los arreglos se utilizan para almacenar las cuatro notas de cada estudiante y para mantener el historial completo de estudiantes evaluados, mientras que los objetos agrupan toda la información relevante de un estudiante (nombre, carrera, notas, promedio, estado, rendimiento, beca, recomendación) en una sola estructura de datos. Las estructuras condicionales (`if/else`, `switch`) permiten clasificar el estado académico, el rendimiento y el porcentaje de beca, y los ciclos (`for`, `forEach`) se emplean para recorrer las notas y el historial al calcular promedios, conteos y estadísticas. Finalmente, Bootstrap se utiliza como framework de estilos para mostrar la información con tarjetas, alertas de color, tablas y un diseño responsivo.

6. Desarrollo paso a paso

5.1 Agregar una cuarta nota al formulario

Primero se obtiene el valor que el usuario escribió en el campo Nota 4 mediante `getElementById()`. Como los datos ingresados desde un formulario son tratados inicialmente como texto, se utiliza la función `Number()` para convertir cada uno en un dato numérico. Finalmente, las cuatro calificaciones se almacenan en un arreglo llamado `notas`, el cual será utilizado posteriormente para calcular el promedio y realizar las demás evaluaciones académicas.

```
<div class="col-md-3">
  <label for="nota4" class="form-label">Nota 4</label>
  <input id="nota4" type="number" class="form-control" min="0"
    max="20" step="0.01">
</div>

const nota4Texto = document.getElementById("nota4").value;
const notas = [Number(nota1Texto), Number(nota2Texto),
  Number(nota3Texto), Number(nota4Texto)];
```

Código 1. Campo Nota 4 en el formulario

5.2 Permitir notas con decimales

Se modificaron los campos de entrada para que acepten valores decimales utilizando el atributo `step="0.01"`. Además, cuando se calcula el promedio del estudiante, el resultado se presenta con dos cifras decimales mediante el método `toFixed(2)`, logrando una presentación más clara y precisa de la información.

```
<input id="nota1" type="number" class="form-control" min="0" max="20"
step="0.01">
{estudiante.promedio.toFixed(2)}
```

Código 2. Configuración de notas con decimales

5.3 Ajustar las carreras del formulario

Se utiliza la etiqueta `<select>` para crear una lista desplegable. Cada opción posee un atributo `value`, el cual es el valor que JavaScript utilizará para identificar la carrera seleccionada y aplicar las condiciones correspondientes en el cálculo de la beca.

```
<select id="carrera" class="form-select">
  <option value="">Seleccione...</option>
  <option value="TI">Tecnologías de la Información</option>
  <option value="Software">Software</option>
  <option value="Sistemas">Sistemas</option>
</select>
```

Código 3. Selector de carreras

5.4 Crear mensaje de beca académica

La función recibe dos parámetros: la carrera y el promedio del estudiante. Mediante una estructura de decisiones `if...else if...else`, verifica cuál de las condiciones se cumple. Cuando encuentra una condición verdadera, devuelve un objeto con tres propiedades:

- **porcentaje:** indica el porcentaje de beca obtenido.
- **texto:** contiene el mensaje que será mostrado al usuario.
- **color:** especifica la clase de Bootstrap que se utilizará para cambiar el color del mensaje.

Si el estudiante no cumple ninguno de los requisitos establecidos, la función devuelve un objeto indicando que no posee beca.

```
function calcularBeca(carrera, promedio) {
  if (carrera === "TI" && promedio > 18) {
    return { porcentaje: 100, texto: "Beca del 100%", color:
      "success" };
  } else if (carrera === "Software" && promedio > 17) {
    return { porcentaje: 80, texto: "Beca del 80%", color:
      "primary" };
  } else if (carrera === "Sistemas" && promedio > 16) {
```

```

        return { porcentaje: 60, texto: "Beca del 60%", color:
"warning" };
    } else {
        return { porcentaje: 0, texto: "Sin beca", color: "secondary" };
    }
}

```

Código 4. Visualización del mensaje de beca

5.5 Mostrar colores según el tipo de beca

Se creó una función que muestra el resultado de la beca en un div. Dependiendo del tipo de beca obtenido, se cambia automáticamente el color del mensaje utilizando las clases de Bootstrap, permitiendo que el usuario identifique fácilmente el resultado.

```

function mostrarBeca(beca) {
    const contenedorBeca = document.getElementById("beca");
    contenedorBeca.className = `alert alert-${beca.color} mt-4`;
    contenedorBeca.classList.remove("d-none");
    contenedorBeca.textContent = `Resultado de beca: ${beca.texto}`;
}

```

Código 5. Colores según el tipo de beca

5.6 Calcular promedio especial sin la nota más baja

Se implementó una función que identifica la nota más baja, la elimina temporalmente del arreglo y calcula un nuevo promedio únicamente con las notas restantes. Esto permite obtener un promedio especial sin modificar las calificaciones originales.

```

function promedioSinNotaBaja(notas) {
    const extremos = evaluarNota(notas);
    const copiaNotas = [...notas];
    const indiceMenor = copiaNotas.indexOf(extremos.menor);
    copiaNotas.splice(indiceMenor, 1);
    return calcularPromedio(copiaNotas);
}

```

Código 6. Cálculo del promedio sin la nota más baja

5.7 Mostrar nota más alta y nota más baja

La función recorre el arreglo de notas y compara cada valor para encontrar la calificación mayor y la menor. Al finalizar el recorrido devuelve ambos valores para mostrarlos en los resultados del estudiante.

```
function evaluarnota(notas) {
  let mayor = notas[0];
  let menor = notas[0];

  for (const nota of notas) {
    if (nota > mayor) {
      mayor = nota;
    }
    if (nota < menor) {
      menor = nota;
    }
  }
  return { mayor, menor };
}
```

Código 7. Obtención de la nota mayor y menor

5.8 Contar notas aprobadas, en recuperación y reprobadas

Se implementó la función `contarnotas()`. Esta función recorre el arreglo de notas utilizando un ciclo `for...of` y evalúa cada calificación mediante una estructura condicional. Si la nota es igual o superior a 14, incrementa el contador de aprobadas; si se encuentra entre 10 y 13.99, incrementa el contador de recuperación; y si es inferior a 10, aumenta el contador de reprobadas. Al finalizar el recorrido, la función devuelve un objeto con los tres contadores.

```
function contarnotas(notas) {
  let aprobadas = 0;
  let reprobadas = 0;
  let recuperacion = 0;
  for (const nota of notas) {
    if (nota >= 14) {
      aprobadas++;
    } else if (nota >= 10) {
      recuperacion++;
    } else {
      reprobadas++;
    }
  }
  return { aprobadas, reprobadas, recuperacion };
}
```

Código 8. Conteo de notas aprobadas, recuperación y reprobadas

5.9 Generar recomendación académica

Se creó la función `estadoRecomendacionAcademica()`, recibe como parámetro el promedio obtenido y utiliza una serie de condiciones `if...else if...else` para determinar cuál es la recomendación más adecuada. Dependiendo del rango en el que se encuentre el promedio, la función devuelve un mensaje diferente.

```
function estadoRecomendacionAcademica(promedio) {  
  if (promedio >= 18) {  
    return "Mantener el desempeño y apoyar a compañeros";  
  } else if (promedio >= 14) {  
    return "Reforzar temas específicos";  
  } else if (promedio >= 10) {  
    return "Asistir a tutorías y practicar ejercicios";  
  } else {  
    return "Repetir contenidos base y solicitar acompañamiento";  
  }  
}
```

Código 9. Generación de la recomendación académica

5.10 Clasificar el rendimiento académico

La función `clasificarRendimiento()` tiene como propósito asignar un nivel de rendimiento al estudiante según el promedio alcanzado entonces se compara el promedio utilizando varias condiciones consecutivas. Si el promedio es igual o superior a 18, el rendimiento se clasifica como Alto; si está entre 14 y 17.99, se considera Medio; entre 10 y 13.99 corresponde a Básico; y cualquier promedio inferior a 10 se clasifica como Bajo.

```
function clasificarRendimiento(promedio) {  
  if (promedio >= 18) {  
    return "Alto";  
  } else if (promedio >= 14) {  
    return "Medio";  
  } else if (promedio >= 10) {  
    return "Básico";  
  } else {  
    return "Bajo";  
  }  
}
```

Código 10. Clasificación del rendimiento académico

5.11 Crear historial de estudiantes evaluados

Se creó un arreglo global llamado `historialEstudiantes`, el cual almacena la información de cada estudiante evaluado correctamente. Después de generar el objeto estudiante, este se agrega al historial utilizando el método `push()`, permitiendo conservar todos los registros para realizar consultas y estadísticas posteriores.

```
const historialEstudiantes = [];  
historialEstudiantes.push(estudiante);
```

Código 11. Creación del historial de estudiantes

5.12 Validar nombres repetidos

Se creó la función `existeNombreRepetido()`, la cual recibe como parámetro el nombre del estudiante y utiliza el método `some()` para recorrer el historial. Si encuentra un nombre igual, ignorando mayúsculas y minúsculas mediante `toLowerCase()`, devuelve `true`; caso contrario, devuelve `false`, evitando registrar estudiantes duplicados.

```
function existeNombreRepetido(nombre) {  
    return historialEstudiantes.some(est => est.nombre.toLowerCase()  
    === nombre.toLowerCase());  
}
```

Código 12. Validación de nombres repetidos

5.13 Crear botón para listar estudiantes

Se implementó la función `mostrarTablaHistorial()`, que recorre el arreglo `historialEstudiantes` mediante `forEach()`. Por cada estudiante crea una fila de la tabla con su información y la agrega dinámicamente al cuerpo de la tabla, mostrando el historial completo.

```
function mostrarTablaHistorial() {  
    const contenedorTabla =  
document.getElementById("contenedorTabla");  
    const cuerpoTabla = document.getElementById("cuerpoTabla");  
  
    if (historialEstudiantes.length === 0) {  
        mostrarResultado("No hay estudiantes evaluados en el  
historial", "info");  
        return;  
    }  
    cuerpoTabla.innerHTML = "";  
    historialEstudiantes.forEach((est, index) => {
```

```

const fila = document.createElement("tr");

fila.innerHTML = `
    <td>${index + 1}</td>
    <td>${est.nombre}</td>
    <td>${est.carrera}</td>
    <td>${est.notas.join(" - ")}</td>
    <td>${est.promedio.toFixed(2)}</td>
    <td>${est.estado}</td>
    <td>${est.rendimiento}</td>
    <td>${est.beca}</td>`;

cuerpoTabla.appendChild(fila);
});
contenedorTabla.classList.remove("d-none");
}

```

Código 13. Listado de estudiantes en una tabla

5.14 Mostrar cuántos estudiantes han sido evaluados

Se creó la función `estudiantesEvaluados()`, que utiliza la propiedad `length` del arreglo `historialEstudiantes` para obtener el número total de estudiantes registrados. Este valor se muestra posteriormente dentro del panel de estadísticas.

```

function estudiantesEvaluados() {
    return historialEstudiantes.length;
}

```

Código 14. Estudiantes evaluados

5.15 Contar aprobados, recuperación y reprobados del curso

Se creó la función `contarAprobadosRecuperacionReprobados()`, que recorre todo el historial de estudiantes utilizando un ciclo `for...of`. Según el promedio de cada estudiante, incrementa el contador de aprobados, recuperación o reprobados y devuelve los resultados para mostrarlos en las estadísticas.

```

function contarAprobadosRecuperacionReprobados() {
    let aprobados = 0;
    let recuperacion = 0;
    let reprobados = 0;
    for (const est of historialEstudiantes) {
        if (est.promedio >= 14) {
            aprobados++;
        } else if (est.promedio >= 10) {

```

```

        recuperacion++;
    } else {
        reprobados++;
    }
}
return { aprobados, recuperacion, reprobados };
}

```

Código 15. Conteo de estados del curso

5.16 Calcular promedio general del curso

Se creó la función `promedioGeneralTodosEstudiantes()`, que utiliza el método `reduce()` para sumar los promedios de todos los estudiantes registrados. Posteriormente divide el resultado para la cantidad de estudiantes y obtiene el promedio general del curso.

```

function promedioGeneralTodosEstudiantes() {
    if (historialEstudiantes.length === 0) {
        return 0;
    }
    const sumaPromedios = historialEstudiantes.reduce((acc, est) =>
acc + est.promedio, 0);
    return sumaPromedios / historialEstudiantes.length;
}

```

Código 16. Cálculo del promedio general del curso

5.17 Buscar estudiante por nombre

Se implementó la función `buscarEstudiante()`, la cual recibe el nombre ingresado por el usuario y utiliza el método `find()` para buscar un estudiante dentro del historial. Si encuentra una coincidencia, muestra la información del estudiante; de lo contrario, informa que no existe un registro con ese nombre.

```

function buscarEstudiante() {
    const nombreBusqueda = inputBuscar.value.trim().toLowerCase();
    const resultadoBusqueda =
document.getElementById("resultadoBusqueda");

    if (nombreBusqueda === "") {
        resultadoBusqueda.className = "alert alert-warning mt-3";
        resultadoBusqueda.classList.remove("d-none");
        resultadoBusqueda.textContent = "Escribe un nombre para
buscar";
    }
}

```

```

        return;
    }

    const estudiante = historialEstudiantes.find(est =>
est.nombre.toLowerCase() === nombreBusqueda);

    if (estudiante) {
        resultadoBusqueda.className = "alert alert-success mt-3";
        resultadoBusqueda.style.whiteSpace = "pre-line";
        resultadoBusqueda.textContent = `Estudiante encontrado:
${estudiante.nombre}
Carrera: ${estudiante.carrera}
Promedio: ${estudiante.promedio.toFixed(2)}/20
Estado: ${estudiante.estado}
Rendimiento: ${estudiante.rendimiento}
Beca: ${estudiante.beca}`;
    } else {
        resultadoBusqueda.className = "alert alert-danger mt-3";
        resultadoBusqueda.textContent = "Estudiante no encontrado";
    }
    resultadoBusqueda.classList.remove("d-none");
}

```

Código 17. Búsqueda de estudiantes por nombre

5.18 Crear ranking de estudiantes

Se creó la función `generarRanking()`, que realiza una copia del historial utilizando el operador de propagación (...). Después ordena los estudiantes mediante el método `sort()`, comparando sus promedios de mayor a menor para generar el ranking.

```

function generarRanking() {
    const copia = [...historialEstudiantes];
    copia.sort((a, b) => b.promedio - a.promedio);
    return copia;
}

```

Código 18. Generación del ranking de estudiantes

5.19 Mostrar estudiante con mayor y menor promedio

Se creó la función `mejorPeorEstudiante()`, que recorre el historial comparando los promedios de cada estudiante. Durante el recorrido actualiza las variables que almacenan el mejor y el peor promedio, devolviendo ambos resultados al finalizar.

```

function mejorPeorEstudiante() {
    if (historialEstudiantes.length === 0) {
        return { mejor: null, peor: null };
    }
}

```

```

    let mejor = historialEstudiantes[0];
    let peor = historialEstudiantes[0];
    for (const est of historialEstudiantes) {
        if (est.promedio > mejor.promedio) {
            mejor = est;
        }
        if (est.promedio < peor.promedio) {
            peor = est;
        }
    }
    return { mejor, peor };
}

```

Código 19. Estudiante con mayor y menor promedio

5.20 Mejorar el objeto final del estudiante

Se creó el objeto estudiante, el cual almacena toda la información generada durante la evaluación, incluyendo datos personales, notas, promedios, estado, rendimiento, beca y recomendación. Posteriormente, este objeto se convierte al formato JSON mediante `JSON.stringify()`, facilitando la visualización organizada de la información.

```

    const estudiante = {
        nombre: nombre,
        carrera: carrera,
        notas: notas,
        promedio: promedio,
        promedioEspecial: promedioEspecial,
        notaMayor: extremos.mayor,
        notaMenor: extremos.menor,
        estado: estado,
        rendimiento: rendimiento,
        beca: beca.texto,
        recomendacion: recomendacion
    };

    historialEstudiantes.push(estudiante);

    mostrarResultado(construirMensaje(estudiante, cantidad),
    obtenerColorEstado(estado));
    mostrarBeca(beca);
    mostrarJSONLista();
    actualizarPanelEstadisticas();
    console.table(estudiante);
}

```

Código 20. Creación del objeto estudiante en formato JSON

7. Capturas de la página web

Evaluador académico

Ingresa los datos y calcule el estado del estudiante.

Nombre

Anahi Correa

Carrera

Tecnologías de la Información

Nota 1

10,20

Nota 2

14,50

Nota 3

16

Nota 4

5

Evaluar

Limpiar

El estudiante Anahi Correa tiene un promedio de: 11.43/20,
por este motivo tiene un estado: Recuperación y un rendimiento: Básico.
La nota más alta es: 16, y la nota más baja es: 5.
Cantidad de notas aprobadas: 2, en recuperación: 1, reprobadas: 1.
Promedio especial (sin la nota más baja): 13.57/20.
Recomendación académica: Asistir a tutorías y practicar ejercicios

Resultado de beca: Sin beca

Estadísticas del curso

1
Evaluados

0
Aprobados

1
Recuperación

0
Reprobados

Promedio general del curso: **11.43**/20
Mejor promedio: Anahi Correa (11.43/20)
Menor promedio: Anahi Correa (11.43/20)

```
[
  {
    "nombre": "Anahi Correa",
    "carrera": "II",
    "notas": [
      10.2,
      14.5,
      16,
      5
    ],
    "promedio": 11.425,
    "promedioEspecial": 13.566666666666668,
    "notaMayor": 16,
    "notaMenor": 5,
    "estado": "Recuperacion",
    "rendimiento": "Básico",
    "beca": "Sin beca",
    "recomendacion": "Asistir a tutorías y practicar ejercicios"
  }
]
```

Ver Historial en Tabla

Ver Ranking

Escribe el nombre del estudiante a buscar...

Buscar

Figura 1. Evaluador Académico

Historial de Estudiantes

#	Nombre	Carrera	Notas	Promedio General	Estado	Rendimiento	Beca
1	Anahi Correa	TI	10.2 - 14.5 - 16 - 5	11.43	Recuperacion	Básico	Sin beca
2	Andrea Sinmaleza	Software	20 - 9.5 - 14 - 10	13.38	Recuperacion	Básico	Sin beca
3	Estefany Briones	Sistemas	20 - 20 - 14 - 14.5	17.13	Aprobado	Medio	Beca del 60%

Figura 2. Historial de estudiantes

Ranking de Estudiantes (mayor a menor promedio)

Posición	Nombre	Carrera	Promedio	Rendimiento
1	Estefany Briones	Sistemas	17.13	Medio
2	Andrea Sinmaleza	Software	13.38	Básico
3	Anahi Correa	TI	11.43	Básico

Figura 3. Ranking de estudiantes

Estefany Briones

Buscar

Estudiante encontrado: Estefany Briones

Carrera: Sistemas

Promedio: 17.13/20

Estado: Aprobado

Rendimiento: Medio

Beca: Beca del 60%

Figura 4. Búsqueda de estudiante por nombre

8. Pruebas realizadas

Estudiante	Carrera	Notas	Promedio	Estado	Rendimiento	Beca
Joel Beltran	TI	19, 20, 19.5, 20	19.63	Aprobado	Alto	100%
Carlos Cedeño	Software	18, 18.5, 17.5, 19	18.25	Aprobado	Alto	80%
Elkin Vera	Sistemas	17.5, 17, 18, 16.5	17.25	Aprobado	Alto	60%
Jhon Herrera	TI	12, 10, 13.5, 11	11.63	Recuperación	Básico	Sin beca
Shirley Alava	Software	8, 6, 9.5, 7	7.63	Reprobado	Bajo	Sin beca

Adicionalmente se comprobó que:

- El sistema rechaza la evaluación cuando algún campo del formulario está vacío (Figura 5).

The screenshot shows a web form titled "Evaluador académico" with the instruction "Ingrese los datos y calcule el estado del estudiante." The form contains the following fields: "Nombre" (text input with "Joel Beltran"), "Carrera" (dropdown menu with "Tecnologías de la Información"), "Nota 1" (text input with "10,50"), "Nota 2" (text input with "16"), "Nota 3" (text input with "10"), and "Nota 4" (empty text input). Below the fields are two buttons: "Evaluar" (blue) and "Limpiar" (white). At the bottom, a yellow message box displays the text "Llena todos los campos".

Figura 5. Validación de datos vacíos

- El sistema rechaza la evaluación cuando se intenta registrar dos veces el nombre "Joel Beltran" (Figura 6).

The screenshot shows the same "Evaluador académico" form as in Figure 5. The "Nombre" field contains "Joel Beltran", "Carrera" is "Tecnologías de la Información", "Nota 1" is "20", "Nota 2" is "16", "Nota 3" is "10,80", and "Nota 4" is "15". The "Evaluar" and "Limpiar" buttons are present. At the bottom, a yellow message box displays the text "Ya existe un estudiante registrado con el nombre 'Joel Beltran'. Ingresa un nombre distinto."

Figura 6. Validación cuando se ingresa el mismo nombre

9. Análisis de resultados

Los resultados obtenidos demuestran que el sistema funciona correctamente al evaluar a los estudiantes según las condiciones establecidas. Por ejemplo, Estefany Briones recibió la beca del 60 % por cumplir con el promedio requerido en la carrera de Sistemas, mientras que Anahi Correa, Andrea Sinmaleza y Joel Beltrán fueron clasificados en estado de Recuperación, sin beca y con la recomendación correspondiente a su desempeño. Además, el cálculo del promedio especial y la validación de nombres repetidos operaron de forma adecuada, garantizando resultados confiables y evitando registros duplicados en el historial.

10. Conclusiones

- Se logró implementar todas las funcionalidades solicitadas, permitiendo evaluar estudiantes, calcular promedios, asignar becas y generar estadísticas de manera correcta, manteniendo un código organizado y fácil de comprender.
- La utilización de funciones, arreglos y objetos permitió distribuir las tareas del programa de forma ordenada, facilitando el desarrollo de nuevas características sin afectar el funcionamiento de las ya existentes.
- Las validaciones implementadas, como el control de notas, campos obligatorios y nombres repetidos, contribuyeron a mantener la información del historial consistente y a obtener resultados confiables en los cálculos y reportes generados.

11. Recomendaciones

- Implementar el almacenamiento del historial, con el fin de conservar la información aunque la página sea cerrada o recargada.
- Incorporar validaciones visuales utilizando las clases de Bootstrap, de manera que el usuario pueda identificar fácilmente los campos que presentan errores antes de realizar la evaluación.
- Dividir el código JavaScript en varios archivos según su función (validaciones, cálculos, historial e interfaz), lo que facilitará el mantenimiento y permitirá ampliar el proyecto con mayor facilidad en el futuro.

12. Criterio personal

Este taller me permitió reforzar la importancia de mantener una estructura organizada para evitar errores al realizar modificaciones. La más complejo de este taller para mí fue la implementación del historial de estudiantes y las validaciones, especialmente al controlar que no se registraran nombres repetidos y al generar el ranking de acuerdo con los promedios obtenidos por que estas funciones requirieron un manejo de arreglos y objetos en JavaScript. Pero también creo que lo más importante del evaluador es el historial de estudiantes, ya que permite almacenar la información, consultar los registros y obtener estadísticas del curso de forma rápida.

13. Bibliografía complementaria

Hedley, J. (n.d.). *jsoup: Java HTML parser, built for HTML editing, cleaning, scraping, and XSS safety*. Jsoup: Java HTML Parser. <https://jsoup.org/>